Crisis Twin

Team work split, API list and Meta model map

Twelve feature pages divided four each among Manan, Aditya and Manthan. The frontend pages already exist with sample data; this plan covers the logic, endpoints and model calls behind them.

Member	Role	Features owned	Effort
Manan	Voice and agents lead	1 Voice war room, 2 Crisis drill, 3 Debate room, 4 Audit trail	16
Aditya	Quant engine lead	5 Crisis Twin simulator, 6 Propagation map, 7 Regret replay, 8 Why it moved	17
Manthan	Data, portfolio and product lead	9 Overview, 10 Holdings and paper trading, 11 Report card, 12 Risk profile, plus data foundation and deployment	10 + 5

Why this split

- **Manan: voice and agents.** Both voice features plus the Debate room and Audit trail share one pipeline: speech, the Meta model client, the agent prompts and logging.
- **Aditya: quant engine.** Simulator, propagation, replay and attribution all use one factor model and one set of historical data, so there is a single version of the maths.
- **Manthan: data, portfolio, product.** Overview, holdings, report and profile sit on the data layer. He also builds the foundation (backend scaffold, database, price loader, API client, deployment) that unblocks everyone early.

```
Manthan: data, portfolio    prices, portfolios, paper trades, database, API scaffold
    |
Aditya: quant engine        factor model, /simulate, /propagate, /replay, /why
    |
Manan: voice and agents     Meta model client, /debate, /voice/intent, audit logging
Every run is saved to the database and logged in the audit trail.
Drill results from Manan feed the risk profile built by Manthan.
```

1. Shared rules (agree on day one)

Owner	Frontend files	Backend areas
Manan	app/(console)/war-room, drill, debate, audit; components/VoiceOrb.js, AuditTable.js, AgentMark.js	voice/, agents/, audit/, llm client
Aditya	app/(console)/simulator, propagation, replay, why; components/viz/ShockGraph.js, Sliders.js	quant/, graph/, backtest/
Manthan	app/(console)/dashboard, holdings, report, profile; lib/api.js	data/, portfolio/, report/, profile/, db, deploy
Shared (ask first)	lib/nav.js, components/ui.js, components/console/Shell.js, app/globals.css	API schemas, config

- **Git.** One branch per feature (feat/manan-war-room). Merge by pull request only. Cross-review: Manan reviews Aditya, Aditya reviews Manthan, Manthan reviews Manan. Never commit keys; use a git-ignored .env.
- **Mock first.** Every endpoint returns the agreed JSON shape on day one (even hard-coded) so the pages integrate immediately. Pages keep lib/sample.js as a fallback. Frontend calls go through one helper, lib/api.js, reading NEXT_PUBLIC_API_URL.

Frozen data shapes

```
// Shock object: output of /scenario/parse, input of /simulate
{ "scenario_name": "Oil spike plus rate hike", "horizon_days": 10,
  "shocks": [ {"factor":"oil_price","change_pct":25}, {"factor":"policy_rate","change_bps":100} ] }
// Simulation result: output of /simulate
{ "run_id": "run-0142", "expected_move_pct": -4.1, "p05": -6.8, "p95": -1.9, "cvar95": -7.4,
  "per_holding": [ { "id": "tata", "weight": 12, "impact_pct": -12.6, "beta": 1.05,
      "breakdown": [ {"factor":"oil_price","value_pct":-5.8}, {"factor":"policy_rate","value_pct":-6.2} ] } ],
  "risk_contribution": [ {"id":"hdfc","share":0.24} ],
  "confidence": "medium", "in_fitted_range": true, "model_version": "factors-v3" }
// Agent message: one per agent turn from /debate
{ "agent": "bear", "text": "...", "confidence": 4,
  "claims": [ {"claim":"...","evidence_field":"per_holding.hdfc.impact_pct","severity":4} ] }
```

2. Every API we use, and where the Meta model fits

2.1 Where the Meta model is used

The Meta model (Muse or another model the hackathon provides) is used for language tasks only: turning words into structured data, arguing in text, and narrating results the code already computed. Confirm the exact model names, endpoint, JSON or tool-calling support, streaming and rate limits in the hackathon documentation, and whether it offers speech; if it does, use it for speech, otherwise use the browser APIs in 2.2.

Feature	Task for the Meta model	Owner	Input -> output	Settings
Simulator	Scenario parser: plain English to shock object	Aditya	User text -> shock JSON (schema-validated)	Temperature 0, JSON output
Voice war room	Intent router: transcript to one intent from a fixed list	Manan	Transcript + last turns -> intent JSON	Temperature 0, JSON output
Voice war room	Spoken summary and daily briefing	Manan	Result JSON -> 2 to 4 spoken sentences	Low temperature, short
Debate room	Bull, Bear, Risk Auditor and Portfolio Manager agents	Manan	Result JSON (+ tool outputs) -> claims with evidence fields	Temperature 0.3 to 0.5, JSON
Crisis drill	Alert wording and debrief sentence	Manan	Scenario + choices -> short text	Low temperature
Why it moved	Story paragraph and plain-language verdict reason	Aditya	Contributions + drivers JSON -> paragraph	Low temperature
Report card	Optional one-line summaries of risk and fix	Manthan	Run JSON -> two sentences	Low temperature
Risk profile	Optional plain-language reading of the profile	Manthan	Scores -> short text	Low temperature

Never use the model for: impact percentages, betas, VaR and CVaR, propagation weights, backtests, the survival score, verdict thresholds or trade execution. Code computes every number; the model only explains or translates them.

Rules for every Meta model call

- **One client.** Manan writes a single LLMClient class with complete(system, user, schema=None), retries, timeouts and JSON parsing. Everyone calls the model only through it, so the model can be swapped by changing environment variables (META_API_KEY, META_MODEL, META_BASE_URL).
- **Keys stay on the server.** Calls go from the FastAPI backend, never from the Next.js browser code.
- **Ground and validate.** Validate JSON against a schema. A grounding check rejects any agent claim whose cited field is missing or whose number does not match the result; retry once, then drop the claim.
- **Control cost and speed.** Send only the fields an agent needs, cache demo scenarios, run independent calls in parallel, and stream the debate so the page and voice can start early.

2.2 External APIs and services

API or service	Used for	Key?	Owner	Notes
Meta model API	All language tasks in 2.1	Yes	Manan (client)	Server side only; confirm limits in hackathon docs
Yahoo Finance via the yfinance library	Daily prices for holdings (NSE tickers end in .NS, for example RELIANCE.NS), crude CL=F, USD/INR INR=X, Nifty 50 ^NSEI, India VIX ^INDIAVIX	No	Manthan	Unofficial, rate limited; cache everything; check terms; verify tickers are current
FRED API (St. Louis Fed)	Backup macro series, for example WTI crude DCOILWTICO and US yields	Free key	Manthan	Optional fallback
RBI and NSE public data	Policy rate history, India VIX history	No	Manthan	CSV download is fine; store raw files
News API, GNews or RSS feeds	Optional headlines for the Why story and daily briefing	Yes	Aditya	Optional; skip if time is short

API or service	Used for	Key?	Owner	Notes
Web Speech API (SpeechRecognition)	Speech to text in the browser	No	Manan	Chrome and Edge; needs localhost or HTTPS
Web Speech API (speechSynthesis)	Speaking, with a different voice per agent	No	Manan	Voices vary by device; test on the demo laptop
Whisper (open source) or a neural TTS service	Optional upgrade for accuracy and nicer voices	Maybe	Manan	Only after the basics work
Vercel and Render, Railway or Fly	Hosting the frontend and the FastAPI backend	Account	Manthan	Docker image for the backend

2.3 Our own endpoints

Endpoint	Owner	Request -> response	Meta model?
POST /scenario/parse	Aditya	text -> shock object	Yes (parser)
POST /simulate	Aditya	shock object, portfolio, optional adjustments -> simulation result	No
POST /propagate	Aditya	shock object -> nodes, edges, paths, hidden exposures	No
GET /replay/{crisis}	Aditya	crisis id -> curves, predicted vs actual, metrics, baselines	No
GET /why?date=	Aditya	date -> contributions, drivers, verdict, story	Yes (story)
POST /debate	Manan	simulation result -> streamed agent messages + recommendation	Yes (4 agents)
POST /voice/intent	Manan	transcript, context -> intent and parameters	Yes (router)
POST /drill/result	Manan	option, seconds, loss at decision -> saved	No
GET /audit/runs, /audit/runs/{id}	Manan	filters -> run list; id -> full run with agent logs	No
GET /portfolio, POST /paper/trade	Manthan	-> holdings and values; confirmed order -> updated holdings	No
GET /report/{run_id}	Manthan	run id -> one-page PDF (and share link)	Optional
GET and POST /profile	Manthan	answers -> profile; profile -> scale and strategy checks	Optional

3. Manan: voice and agents lead

Backbone first (days 1 to 2): the LLMClient, a prompt folder (Bull, Bear, Auditor, Portfolio Manager, intent router, narrator), the grounding validator, and a small browser module for listening and speaking (start, stop, transcript events, speak with a named voice, queue, cancel).

1. Voice war room (/war-room, Manan, effort 5, needs: Aditya /simulate, your /debate)

What it does. Run the product by talking: ask, command, interrupt. Advisors answer in different voices. This is the mandatory voice feature, so reliability matters most.

Build

- In VoiceOrb.js, replace the placeholder toggle with start and stop of speech recognition. Use push-to-talk for the demo and show interim text under the orb.
- Send the final transcript to /voice/intent. The Meta model picks one intent from a fixed list (RUN_SCENARIO, START_DEBATE, INTERRUPT_AGENT, REBALANCE, PAPER_TRADE, EXPLAIN_MOVE, BRIEFING, DRILL_DECISION) and returns JSON; validate it before acting.
- Dispatch: parse then simulate for scenarios, /debate for debates, /paper/trade for trades. Any trade needs spoken confirmation first.
- Speak a 2 to 4 sentence summary, not the full result. Give each agent its own voice, pitch and rate; queue utterances so they never overlap.
- Barge-in: when speech is detected during playback, cancel synthesis at once and route the new command with the debate context so "Bear, respond to that" works.
- Keep the conversation in React state and wire the typed input to the same router as a fallback. Add a BRIEFING intent using /portfolio and /why.

```
User: "What if oil jumps twenty five percent and Bear thinks my banks are safe?"
 -> {"intent":"RUN_SCENARIO","scenario_text":"oil +25%","post_action":"START_DEBATE","focus_agent":"bear"}
```

Meta model and APIs. Meta model: intent router, spoken summary and briefing. Browser SpeechRecognition and speechSynthesis for speech. Calls every internal endpoint.

Done when. a spoken oil-and-rates scenario returns a spoken summary in about 5 seconds, interruption works, and typed input behaves the same.

Watch out. Speech recognition needs Chrome or Edge on localhost or HTTPS. Indian company names are misheard, so keep an alias list. Test the demo room and record a backup video.

2. Crisis drill (/drill, Manan, effort 3, needs: Voice war room, Aditya /simulate)

What it does. A timed rehearsal: breaking alert, countdown, a decision by voice or tap, the outcome, then a debrief comparing stated tolerance with real behaviour.

Build

- Use a reducer state machine: ALERT, COUNTDOWN, DECISION, OUTCOME, DEBRIEF. Keep drills in a JSON library (alert text, shock object, time limit, difficulty).
- Speak the alert with a newsroom voice. Drive the Ring value and the 00:42 label from one timer; clear it on unmount. A timeout is recorded as "no decision".
- Accept a tap or a spoken DRILL_DECISION. Re-run /simulate with the chosen adjustment and show before and after in the live chart.
- Debrief: compare the stated tolerance from /profile with the loss at which the user acted. POST the result to /drill/result for Manthan's profile.

Meta model and APIs. Meta model: alert wording and the debrief sentence (short text). Speech APIs as in feature 1. Calls /simulate, /profile, /drill/result.

Done when. a full drill takes under two minutes, voice and tap both work, and results appear in the risk profile.

Watch out. Timers that keep running after leaving the page. Do not reveal the right answer in the alert.

3. Debate room (/debate, Manan, effort 5, needs: Aditya /simulate and /propagate)

What it does. Bull, Bear and Risk Auditor argue over a simulation result; the Portfolio Manager recommends, and dissent stays on the record.

Build

- Build the graph in LangGraph (or plain functions): Bull, then Bear (sees Bull), then Auditor (sees both plus raw numbers), then Portfolio Manager. One optional rebuttal round.
- Each agent gets the result JSON and must reply in JSON (claims, evidence fields, severity, confidence) using only supplied numbers.
- Give the Auditor tools backed by Aditya's code: top_risk_contributors(), correlation_in_stress(), concentration(), so hidden exposures are computed.
- Run every output through the grounding validator. Stream each message by server-sent events so the page and the voice layer can start immediately.
- The recommendation's before and after numbers come from a re-run of /simulate with the adjustments, never from the model. Feed the track-record panel from Aditya's replay results.

Meta model and APIs. Meta model: all four agents (the heaviest use). Calls /simulate (re-run), /propagate, replay results for the scoreboard.

Done when. three grounded agents run on a real /simulate result, every recommendation number matches a re-simulation, and the page streams the debate.

Watch out. Models invent numbers, so the validator is mandatory. Chains are slow: keep prompts short and cache demo scenarios. Tell the Bear to attack the strongest claim so agents do not simply agree.

4. Audit trail (/audit, Manan, effort 3, needs: Manthan's database, every endpoint)

What it does. Logs every run with inputs, each agent's reasoning, model version and confidence, so any recommendation can be traced.

Build

- Add middleware that writes runs (inputs, outputs, model version, confidence, time) and agent_logs (agent, prompt, output, latency) whenever /simulate or /debate runs. Manthan creates the tables.
- Endpoints: GET /audit/runs (filters) and /audit/runs/{id}. Pass real rows into AuditTable.js in place of the sample rows.
- Fill the Explain this number panel from the per-holding breakdown in the result. Set High, Medium or Low confidence from Aditya's in_fitted_range flag and data coverage. Add a JSON export button.

Meta model and APIs. No Meta model calls; it stores prompts and outputs from the calls others make.

Done when. every run appears within a second, expanding a row shows parsed shocks and the agent log, and the confidence filter works.

Watch out. Logging must never block or crash a simulation: write in the background with try/except, and cap prompt size.

4. Aditya: quant engine lead

Backbone first (days 1 to 3): use Manthan's price loader for holdings and factors (crude, USD/INR, bond yield or policy rate, India VIX, Nifty 50). Fit each holding's returns on factor returns over a rolling 3 to 5 year window to get betas and the residual covariance, versioned (factors-v1). Put the maths in a pure Python package with unit tests (apply_shock, monte_carlo, var_cvar, propagate, backtest); the API is a thin wrapper.

5. Crisis Twin simulator (/simulator, Aditya, effort 5, needs: Manthan's data, Manan's LLM client)

What it does. The core feature: describe a crisis in words or sliders and get the expected move, a likely range, before and after value, and impact per holding.

Build

- POST /scenario/parse sends the text to the Meta model with the shock schema; validate with Pydantic and reject unknown factors; fall back to sliders on failure.
- Expected asset impact is the sum of beta times shock per factor, with a stress adjustment because correlations rise in crises. Run about 5,000 Monte Carlo draws for the 5th and 95th percentile and CVaR at 95%. Fix the random seed.
- Set confidence from two checks: is the shock inside the factor's historical range, and how much data backs each beta. Return the full result object including per-holding breakdowns.
- Wire the page: textarea and chips to /scenario/parse, sliders and horizon lifted to page state (Sliders.js and Segmented need onChange), results panels from the response, loading and error states. Accept an optional list of weight adjustments for the debate and drill.

Meta model and APIs. Meta model: scenario parser only. The impact numbers come entirely from the factor model.

Done when. the same input gives the same result, ranges widen for large shocks, confidence drops outside the historical range, and unit tests cover the maths and parser validation.

Watch out. Never let the model output impact percentages. Betas change in crises, so state that as a limit. Use at least 3 years of data.

6. Propagation map (/propagation, Aditya, effort 4, needs: Your betas, Manthan's sector data)

What it does. Shows how a shock travels from cause to sectors to holdings, including indirect paths a sector view would miss.

Build

- Nodes: shock factors, sectors, holdings. Edges: sector and holding betas plus a hand-curated CSV of 30 to 60 indirect links (bank lending to auto makers, fuel to airlines).
- Push impact from the shocked node with a decay per hop (about 0.6) for up to four hops; keep every path. Flag holdings that receive impact through several indirect paths as hidden exposures with a plain note.
- Return nodes, edges, top paths and hidden exposures that agree with /simulate. Change ShockGraph.js to take this payload as props instead of lib/sample.js and fetch per scenario tab.

Meta model and APIs. No Meta model calls. Hidden-exposure notes are written from your curated CSV; the Auditor agent reads your output.

Done when. changing the scenario redraws the graph with real numbers and hidden exposures match what the Auditor reports.

Watch out. Curated links are assumptions, so label them. Hide weak edges to keep the graph readable.

7. Regret replay (/replay, Aditya, effort 5, needs: Your simulator, Manthan's history)

What it does. Re-runs real crashes (2008, 2020, 2022, an oil spike) with and without the system's suggestions. This is your proof of credibility.

Build

- Define each crisis window. Fit betas only on data before the start date (out-of-sample) to avoid look-ahead bias; predict the loss, then compare with the actual loss and show the miss honestly.
- Apply the recommended adjustments on day 0 with transaction costs and slippage. Compute baselines (equal weight, 60/40, buy-and-hold index) and max drawdown, Sharpe, Sortino and Calmar.
- Cache results; wire LineChart and the calibration panel to GET /replay/{crisis}. Also expose per-agent hit rates for Manan's scoreboard.

Meta model and APIs. No Meta model calls.

Done when. each crisis shows real curves and the low point for both strategies, calibration shows predicted versus actual, and a short methods note explains the out-of-sample rule.

Watch out. Look-ahead and survivorship bias will be spotted by reviewers. Do not tune adjustments on the crisis you show.

8. Why it moved (/why, Aditya, effort 3, needs: Manthan's portfolio data, Manan's LLM client)

What it does. Explains a day's move by holding and by cause, tells the story in plain language, and gives a verdict: noise, temporary shock or structural risk.

Build

- Contribution by holding is weight times that day's return; the bars must sum to the portfolio move. Attribute drivers by regressing portfolio returns on a few factor returns and report the residual.
- Verdict rules: Noise if within 1.5 standard deviations of normal days; Temporary if driven by a factor that historically reverts within days; Structural if the same driver pushed several windows the same way.
- Pass the computed numbers to the Meta model for a short story, then run the grounding check. Wire the date options (Today, Worst day this month, Pick a date) to a date parameter.

Meta model and APIs. Meta model: the story paragraph only. Optional news API for headline context.

Done when. bars add up to the headline move, the verdict changes sensibly between a quiet and a crash day, and the story never uses a number that is not in the data.

Watch out. Correlated factors can double count. Handle market holidays when finding the worst day.

5. Manthan: data, portfolio and product lead

Foundation first (days 1 to 3), it unblocks everyone: a FastAPI scaffold with CORS, health check, .env loader and stub endpoints that return the agreed JSON; a database (PostgreSQL, or SQLite for the hackathon) with migrations for assets, prices, factors, portfolios, holdings, runs, agent_logs, paper_trades, profiles and drill_results; a price and factor loader (yfinance, with FRED or RBI and NSE files as backups) that cleans and caches data; lib/api.js with fallback to sample data; and Docker plus a hosting plan.

9. Overview (/dashboard, Manthan, effort 2, needs: Portfolio data, /simulate, audit)

What it does. Home screen: portfolio value, today's move, last crisis result, survival score and recent runs.

Build

- Value and today's move from /portfolio; last crisis, confidence and recent runs from /audit/runs.
- Define the survival score and document it: score = 100 - 100 * min(1, abs(cvar95) / risk_budget), where risk_budget is the user's loss limit (default 10%). The report card uses the same value.
- Show biggest movers from the last run, weight by sector, and quick-start shock links that open the simulator preselected.

Meta model and APIs. No Meta model calls.

Done when. real numbers load, the page falls back to sample data if the API is down, and the survival score matches the report.

Watch out. Show loading skeletons, not empty panels; this is the first page judges open.

10. Holdings and paper trading (/holdings, Manthan, effort 4, needs: Your data foundation)

What it does. Shows what the portfolio owns, how each holding behaves under stress, and lets the user place simulated trades.

Build

- GET /portfolio returns quantity, latest price, value, weight and day change from the price cache; replace the sample HOLDINGS and show a last-updated time.
- POST /paper/trade validates side, quantity and price, updates holdings, writes paper_trades and requires confirmed: true (set by the page or by Manan's spoken confirmation).
- Make the ticket interactive (Segmented and inputs need state and submit) with plain success and error messages; compute the concentration note.

Meta model and APIs. No Meta model calls. Data from yfinance and backup sources.

Done when. a paper trade changes the weights and appears in the audit trail, and invalid orders are rejected clearly.

Watch out. Data APIs rate limit, so cache. Keep the Simulated tag and disclaimer visible; never imply real trading.

11. Report card (/report, Manthan, effort 2, needs: /simulate result, Manan's recommendation)

What it does. A one-page summary that can be downloaded or shared.

Build

- Load the selected run into the sheet: scenario, survival score, expected move, range, after-fix value, hidden risk and fix.
- GET /report/{run_id} builds a one-page PDF (ReportLab or an HTML template) matching the preview, with the disclaimer. Add a signed, expiring share token for a read-only page.

Meta model and APIs. Meta model optional: two summary sentences from the run JSON.

Done when. the PDF matches the preview and the share link works in a private window.

Watch out. Use built-in PDF fonts to avoid missing glyphs; keep private details out of shareable links.

12. Risk profile (/profile, Manthan, effort 2, needs: Manan's drill results, Aditya's /simulate)

What it does. Shows how much loss the user can really stomach, from answers and from drill behaviour, and checks strategies against that limit.

Build

- Store the four questions in JSON and map each answer to a loss threshold. Compute a comfort level and a likely-to-sell point; move the breaking point closer to zero when drills show earlier selling than stated.
- Classify reaction style by simple rules (sells early, holds then sells late, holds through, freezes). For each strategy compare its worst case from /simulate or /replay with the breaking point to give Fits, Stretch or Too risky.
- GET and POST /profile; render the scale and table from the response. Manan's debrief reads the stated tolerance from here.

Meta model and APIs. Meta model optional: a short plain-language reading of the profile.

Done when. changing an answer moves the markers, a contradicting drill changes the reaction style, and verdicts match the thresholds.

Watch out. This is not a regulatory suitability assessment; say so on the page. Keep profiles per user and out of the audit log.

6. Timeline, handoffs and risks

Phase	Manan	Aditya	Manthan	Handoff
1. Foundation (days 1-3)	LLM client, prompts, grounding check, speech helpers	Factor model, core library, mock /simulate	Scaffold with stub endpoints, DB, price loader, lib/api.js	Stubs published; Aditya gets prices; team gets LLM client
2. Features on mocks (4-8)	Debate agents, intent router, war room wiring	Parser, simulator, propagation, replay engine	Portfolio, paper trades, dashboard, report PDF	JSON shapes frozen
3. Real integration (9-12)	Debate on real /simulate, voice end to end, audit logging	Replay results, why it moved, confidence	Profile, share links, deployment	Speak scenario, see map, hear debate, get report
4. Drill and polish (13-15)	Crisis drill, audit export	Calibration, baselines, tests	Docker, README, CI, loading and error states	Drill results feed profile
5. Demo prep (16-18)	Voice rehearsal, backup video	Methods note, interview Q and A	Deployment check, demo data	Three full rehearsals; cut flaky parts

- **Daily:** a 10-minute check-in (done, blocked, need from others) and one shared integration test: type or speak the oil-and-rates scenario and confirm the whole chain. Freeze scope two days before the demo.
- **Demo roles:** Manthan opens with the problem, overview and report card; Aditya speaks the crisis, shows the map and the numbers; Manan runs the voice debate with an interruption and shows the audit trail; each answers questions in their own area.
- **Done for every feature:** works on the demo laptop, intentional loading and error states, a README note with endpoints and known limits, a test for the core logic, reviewed by the assigned teammate.

Risk	Owner	Mitigation
JSON shapes drift	All	Freeze the three shapes on day one; changes need all three to agree
Voice fails in the demo room	Manan	Push-to-talk, typed fallback, backup video, test the room early
Model invents numbers	Manan, Aditya	Model never computes impact; grounding validator on every output
Backtest looks too good	Aditya	Out-of-sample fit, baselines, report misses, cross-review
Data API limits or gaps	Manthan	Cache, keep raw downloads, fall back to sample data
Meta API limits or downtime	Manan	Cache demo scenarios, timeouts, retry once, canned fallback text
Uneven workload	All	Re-balance after phase 2; swap two features of similar effort if needed

Assignments are a proposal based on skills and dependencies. Verify all Meta and third-party API details in their official documentation before building.